Python 金融数据分析

第 4 章　*pandas* 数据分析

主讲人：　王伟冠, 吕文俊
院　系：　经济学院金融系

上海大学

2026 年 10 月 17 日



目录

1. 初识 pandas
2. Series：带标签的一列数据
3. DataFrame：金融数据的"主战场"
4. 数据选取与筛选
5. 宽表与长表：结构与转换
6. 排序与分组
7. 数据清洗基础与缺失值
8. 数据合并与变形进阶
9. 金融综合应用：行情与样本检查
10. 金融时间序列
11. 读写数据
12. 综合案例与小结



1. 初识 pandas

教学路线：可分四个单元推进

1. 表与索引：选择、赋值、对齐，再用股票行情理解宽长表转换与分组。
2. 清洗：类型、重复、缺失、异常，保留处理依据。
3. 合并与变形：连接关系、行数变化、重复键聚合与多层索引。
4. 金融应用：复用股票行情，检查覆盖范围、计算收益率并选择可比样本。

课堂节奏

每单元可安排一周，也可将讲解、上机和讲评分开。完成本单元练习后再推进，不要求一周讲完整章。

配套 Notebook：按主题练习

- 全章使用 `04_pandas.ipynb`，小节顺序与课件一致。
- 第 1–5 节：表、索引、筛选与宽长表转换。
- 第 6–8 节：分组、清洗、合并与变形进阶。
- 第 9–11 节：行情样本检查、时间序列与读写数据。
- 第 12 节：成绩清洗综合练习与三股风险统计。

运行方式

选择课程 `.venv` 内核，从头按顺序运行；练习答案放在折叠单元格中，先动手再展开。

- 理解 Series 与 DataFrame 两种核心结构
- 掌握数据的选取：按列、按标签 `loc`、按位置 `iloc`、条件筛选
- 掌握排序与分组聚合（`groupby`）
- 掌握金融时间序列的基本操作：日期索引、`pct_change`、重采样
- 会处理缺失值，会用 `read_csv` 读入真实数据
- 掌握清洗、连接、长宽表转换与组内统计，能够对一张股价表完成"收益率—风险—相关性"的完整分析

前置知识

第 2 章的字典/列表、第 3 章的 NumPy——pandas 就建在 NumPy 之上。

NumPy 不够用了?

- 第 3 章我们算出了收益率数组，但马上会遇到新问题：
  - 每个数字对应哪一天？——NumPy 数组没有标签；
  - 行情数据里有日期、代码、名称——普通数组通常使用统一的数据类型；
  - 数据有缺失（停牌）怎么办？
  - 从 Excel/CSV 读进来的表格如何直接分析？
- **pandas** 就是为此而生：给数组加上行列标签，成为"数据表"。

一个直观比喻

NumPy 数组像一张写满数字的草稿纸；
pandas DataFrame 像一张带表头和日期列的 Excel 表，而且能用代码操作。

```python
import numpy as np
import pandas as pd          # 约定俗成，照抄
```

结构	可以理解为	金融中的例子
`Series`	一列带索引的数据	某股票的收盘价序列
`DataFrame`	一张带行列标签的表	多股票 × 多日期的价格表

- DataFrame 本质上就是"共享同一个索引的若干 Series"。
- 本章几乎所有内容都围绕这张"表"的查看、选取、变形与统计。

2. Series：带标签的一列数据

```
s = pd.Series([1500.35, 128.60, 55.20],
              index=["600519", "000858", "601318"])
s
```

```
600519    1500.35
000858     128.60
601318      55.20
dtype: float64
```

- 左边是**索引**（index，相当于标签），右边是值。
- 不指定索引时，默认为 0, 1, 2, …（像列表）。
- 索引可以是股票代码、日期、任何你需要的标签。

按标签取值与向量化

```
s["600519"]              # 1500.35  按标签取值
s[["600519", "601318"]]  # 取多个

s * 1.05                 # 所有值涨5%（向量化，继承自NumPy）
s[s > 100]               # 筛选出大于100的
```

- Series 支持 NumPy 式向量化运算与布尔筛选——旧知识直接复用。
- 取值时用的是标签而不是位置：`s["600519"]`。

关键概念：对齐

两个 Series 运算时按**索引对齐**，而不是按位置：
" 茅台的价格" 只和" 茅台的权重" 相乘，顺序打乱也不会错。

关键概念：对齐——索引顺序不同怎么办？

两位同学的基础分与加分分别记录如下，每个标签代表一位同学。
问题：加分表的顺序反了，怎样把加分加给对应的同学？

```python
base = pd.Series([10, 20], index=["A", "B"])
bonus = pd.Series([1, 2], index=["B", "A"])
```

pandas：按索引标签相加

```python
print(base + bonus)
# A    12
# B    21
# dtype: int64
```

A 找到 A：$10 + 2 = 12$；
B 找到 B：$20 + 1 = 21$。

NumPy：按数组位置相加

```python
base_np = np.array([10, 20])
bonus_np = np.array([1, 2])
print(base_np + bonus_np)
# [11 22]
```

第一个加第一个：$10 + 1 = 11$；
第二个加第二个：$20 + 2 = 22$。

顺序与标签

NumPy 按位置运算，需要事先统一数据顺序；pandas **自动按相同的索引标签匹配**，即使排列顺序不同，也能正确相加。

Series 对齐：缺少一个索引标签会怎样？

沿用同学分数的例子：基础分 Series 有 A、B、C，
加分 Series 只有 B、A。**相加时，找不到对应加分的 C 会得到什么？**

```
base = pd.Series([10, 20, 30],
                 index=["A", "B", "C"])
bonus = pd.Series([1, 2],
                  index=["B", "A"])
```

```
print(base + bonus)
# A    12.0
# B    21.0
# C     NaN
# dtype: float64
```

自动匹配后的计算

- A：$10 + 2 = 12$。
- B：$20 + 1 = 21$。
- C：基础分为 30，但没有对应加分，结果为 `NaN`（缺失值）。

自动对齐不等于自动补零

两个 Series 相加时**自动按索引标签匹配**，保留双方出现的标签；缺少对应值时，结果为 `NaN`，不会把缺失的加分自动当成 0。

3. DataFrame：金融数据的”主战场”

从字典创建 DataFrame

```
data = {"茅台": [1500.0, 1520.0, 1515.0],
        "五粮液": [128.0, 129.5, 127.0],
        "平安": [55.0, 54.5, 56.0]}
df = pd.DataFrame(data,
                  index=["2026-07-01", "2026-07-02", "2026-07-03"])
df
```

- 字典的**键**变成列名，列表里的值成为该列数据。
- 约定俗成的金融数据方向：行 = 日期，列 = 资产。
- Notebook 中直接写变量名 `df`，会以漂亮的表格显示。

沿用前面的 df：每行是一个交易日，每列是一只股票，数值为教学示例中的收盘价（元）。

```
print(df.shape)
# (3, 3)
print(df.columns)
# Index(['茅台', '五粮液', '平安'], dtype='object')
print(df.index)
# Index(['2026-07-01', '2026-07-02', '2026-07-03'],
#       dtype='object')
```

- shape 表示（行数，列数）：3 个交易日、3 只股票。
- columns 是股票名称；index 是日期标签。
- 这里日期以字符串创建，因此索引类型是 object，还不是日期时间索引。

查看股票数据：前几行与后几行

`head(2)` 查看前两行；`tail(1)` 查看最后一行。括号中的数字指定行数，省略时默认取 5 行。

```
1 print(df.head(2))
```

	茅台	五粮液	平安
2026-07-01	1500.0	128.0	55.0
2026-07-02	1520.0	129.5	54.5

```
1 print(df.tail(1))
```

	茅台	五粮液	平安
2026-07-03	1515.0	127.0	56.0

输出保留原来的日期索引与股票列名，不改变 `df`。

查看股票数据：类型与非缺失数量

问题：各列是什么类型？每列有多少个非缺失值？

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 3 entries, 2026-07-01 to 2026-07-03
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   茅台      3 non-null      float64
 1   五粮液     3 non-null      float64
 2   平安      3 non-null      float64
dtypes: float64(3)
memory usage: 96.0+ bytes
```

- 3 行数据，每列都有 3 个非缺失值：本例没有缺失价格。
- 三列均为 `float64`，可以进行数值计算。
- `info()` 直接打印概要，不需要外套 `print()`。

```
1 print(df.describe().round(2))
```

	茅台	五粮液	平安	含义
count	3.00	3.00	3.00	非缺失观测数
mean	1511.67	128.17	55.17	均值
std	10.41	1.26	0.76	样本标准差
min	1500.00	127.00	54.50	最小值
25%	1507.50	127.50	54.75	下四分位数
50%	1515.00	128.00	55.00	中位数
75%	1517.50	128.75	55.50	上四分位数
max	1520.00	129.50	56.00	最大值

- 每只股票分别沿日期方向统计；`round(2)` 将结果四舍五入到两位小数。
- 茅台这 3 天的平均收盘价为 1511.67 元。
- 这里统计的是**价格**；价格标准差不是收益率波动率，3 天数据也不足以评价长期风险。

一列与一列表：维度不同

同样取出茅台价格，**一个列名**与**列名列表**得到的结构有什么不同？

```python
one_column = df["茅台"]
one_table = df[["茅台"]]
print(type(one_column), one_column.shape)
# <class 'pandas.core.series.Series'> (3,)
print(type(one_table), one_table.shape)
# <class 'pandas.core.frame.DataFrame'> (3, 1)
```

```python
print(one_column)
# 2026-07-01    1500.0
# 2026-07-02    1520.0
# 2026-07-03    1515.0
# Name: 茅台, dtype: float64
```

```python
print(one_table)
#                茅台
# 2026-07-01  1500.0
# 2026-07-02  1520.0
# 2026-07-03  1515.0
```

(3,) 是一维 Series；(3, 1) 是三行一列的二维 DataFrame。两者都保留日期索引。

4. 数据选取与筛选

```
df["茅台"]                # 取一列 -> Series
df[["茅台", "平安"]]      # 取多列 -> DataFrame
df["2026-07-01"]          # 报错！方括号默认是"按列"
```

- `df[...]` 里放列名：取**列**。
- 想按行取，用切片（按位置）或下一页的 `loc/iloc`。

```
df[0:2]             # 前两行（位置切片，含头不含尾）
```

最常见的困惑

为什么 `df["茅台"]` 可以，`df["2026-07-01"]` 却报错？
因为 `[]` 默认针对**列**。规则记牢，少查一半报错。

loc：按标签选；iloc：按位置选

```python
df.loc["2026-07-02"]                # 该行全部列
df.loc["2026-07-02", "茅台"]        # 指定行、指定列 -> 1520.0
df.loc[:, "茅台"]                   # 所有行、茅台列

df.iloc[0]                          # 第0行（按位置）
df.iloc[0:2, 1]                     # 前2行、第1列
df.iloc[-1]                         # 最后一行
```

- loc：用**标签**（日期、列名）；写法 [行，列]。
- iloc：用**位置**（0, 1, 2...）；与列表索引规则相同。
- 先想清楚"我要按名字找还是按位置找"，再选工具。

条件筛选：选出满足条件的行

```python
df[df["茅台"] > 1510]          # 茅台价格高于1510的日子
df[(df["茅台"] > 1500) & (df["平安"] > 55)]
```

- 与 NumPy 布尔索引完全一致：先算出布尔列，再用来筛行。
- 多条件组合用 &、|，每个条件加括号。
- 金融例子：涨幅超过 5% 的日子、PE 低于 10 且股息率高于 4% 的股票。

课堂练习

用一行代码选出"五粮液价格低于 128"的所有交易日。

沿用三只股票的价格表。**问题**：怎样让 pandas 把索引理解为日期，并支持按月选取？

```
df.index = pd.to_datetime(df.index)
df = df.sort_index()
print(type(df.index))
# <class 'pandas.core.indexes.datetimes.DatetimeIndex'>
print(df.index.strftime("%Y-%m-%d").tolist())
# ['2026-07-01', '2026-07-02', '2026-07-03']
```

- `pd.to_datetime` 将日期字符串解析成日期时间，索引成为 `DatetimeIndex`。
- `sort_index()` 按日期升序排列；后面都在这个唯一、有序的日期索引上操作。
- `strftime` 仅用于格式化显示，`df.index` 仍是日期时间索引。

问题：取 7 月 1 日到 3 日的茅台价格，与取第 0、1 个位置，有什么不同?

loc：按日期范围

```
print(df.loc["2026-07-01":"2026-07-03",
             "茅台"])
# 2026-07-01    1500.0
# 2026-07-02    1520.0
# 2026-07-03    1515.0
# Name: 茅台, dtype: float64
```

包括起止日期，得到 3 行。

iloc：按当前位置

```
print(df.iloc[0:2, 0])
# 2026-07-01    1500.0
# 2026-07-02    1520.0
# Name: 茅台, dtype: float64
```

包括位置 0，不包括位置 2，得到 2 行。

日期范围不是行数

`loc` 根据时间标签确定范围；`iloc` 根据当前行的位置取值。日期切片只返回已有记录，不会补出周末、节假日或缺失日期。

按年、按月与开放区间选取

有了 DatetimeIndex，可以只写年份或年月，选取该时间段的全部已有记录。

```
print(df.loc["2026-07", "茅台"])
# 2026-07-01    1500.0
# 2026-07-02    1520.0
# 2026-07-03    1515.0
# Name: 茅台, dtype: float64
print(df.loc[:"2026-07-02", "茅台"])
# 2026-07-01    1500.0
# 2026-07-02    1520.0
# Name: 茅台, dtype: float64
```

- df.loc["2026", "茅台"] 取全年；本例仅有这 3 天，因此结果与按月选取相同。
- :"2026-07-02" 表示从开头到该日，包含该日。
- "2026-07-02": 表示从该日到末尾，本例得到 1520.0、1515.0 两条记录。

条件筛选与赋值：标记高价交易日

问题：怎样标记茅台收盘价高于 1510 元的日期？在副本中增加标记列，价格不变。

```
marked = df.copy()
marked["高价日"] = False
mask = marked["茅台"] > 1510
marked.loc[mask, "高价日"] = True
print(marked[["茅台", "高价日"]])
```

	茅台	高价日
2026-07-01	1500.0	False
2026-07-02	1520.0	True
2026-07-03	1515.0	True

一次指定行与列

`marked.loc[mask, "高价日"] = True` 明确修改哪些行、哪一列；避免使用 `marked[mask]["高价日"] = True` 这样的链式赋值。

沿用日期索引的股票价格表 `df`：

1. 选取 2026 年 7 月 2 日至 3 日的五粮液、平安价格。
2. 在这段日期内，筛选五粮液价格低于 128 元的记录。
3. 在 `df` 的副本中增加“关注日”列，将满足上述条件的日期标记为 `True`，其余为 `False`。

检查思路

先手工确定日期范围，再检查价格条件；比较时间切片和布尔筛选各自保留了哪些行。

5. 宽表与长表：结构与转换

将前面的行情扩展到 6 个交易日：**行是日期，列是股票**，收盘价单位为元。

```
dates = pd.to_datetime([
    "2026-07-01", "2026-07-02", "2026-07-03",
    "2026-07-06", "2026-07-07", "2026-07-08"])
prices_wide = pd.DataFrame({
    "茅台": [1500, 1520, 1515, 1530, 1525, 1545],
    "五粮液": [128, 129.5, 127, 130, 129, 131.2],
    "平安": [55, 54.5, 56, np.nan, np.nan, np.nan]},
    index=dates)
prices_wide.index.name = "日期"
print(prices_wide.shape)
# (6, 3)
```

- 教学合成数据；日期索引是 `DatetimeIndex`，可继续用 `loc` 按时间切片。
- 平安后 3 天没有报价，用 `NaN` 表示未知，不填 0。
- `index.name` 只是给索引起名，便于后面将日期转为普通列。

查看宽表：先看同一天的不同股票

```
1 print(prices_wide)
```

日期	茅台	五粮液	平安
2026-07-01	1500	128.0	55.0
2026-07-02	1520	129.5	54.5
2026-07-03	1515	127.0	56.0
2026-07-06	1530	130.0	NaN
2026-07-07	1525	129.0	NaN
2026-07-08	1545	131.2	NaN

- 宽表便于按日期查看多只股票，也便于逐列计算均值、收益率。
- `prices_wide.mean()` 就能得到各只股票的有效价格均值，不需要先分组。

为什么还要转成长表?

接下来要逐只股票遍历行情、按记录数筛选整只股票，并应用组内函数。把股票名放进一列，就可以用统一的 `groupby("股票")` 写法组织这些操作。

从宽表到长表：把股票列展开为记录

```python
long_prices = prices_wide.reset_index().melt(
    id_vars="日期", var_name="股票", value_name="收盘价")
print(long_prices.shape)
# (18, 3)
market = long_prices[["股票", "日期", "收盘价"]].dropna(
    subset=["收盘价"])
print(market.shape)
# (15, 3)
```

1. `reset_index()`：把日期索引变成“日期”列。
2. `melt`：保留日期，将股票列名放入“股票”列，将对应数值放入“收盘价”列。
3. 6 个日期 $\times$ 3 只股票展开为 18 行；`market` 仅保留其中 15 条有报价的记录。

明确本例的筛选口径

`prices_wide` 和 `long_prices` 保留缺失信息；`market` 用于后续有效报价的分组分析。删除缺失价格行不等于补齐数据，也不表示对应日期价格为 0。

查看 market：15 条行情记录

```
print(market)
```

```
    股票          日期     收盘价
0    茅台  2026-07-01  1500.0
1    茅台  2026-07-02  1520.0
2    茅台  2026-07-03  1515.0
3    茅台  2026-07-06  1530.0
4    茅台  2026-07-07  1525.0
5    茅台  2026-07-08  1545.0
6   五粮液  2026-07-01   128.0
7   五粮液  2026-07-02   129.5
8   五粮液  2026-07-03   127.0
9   五粮液  2026-07-06   130.0
10  五粮液  2026-07-07   129.0
11  五粮液  2026-07-08   131.2
12   平安  2026-07-01    55.0
13   平安  2026-07-02    54.5
14   平安  2026-07-03    56.0
```

- 左侧 0–14 是自动生成的行索引。
- 每行对应一只股票在一天的收盘价，单位为元。
- 同一日期可以出现多次，因为对应不同股票。
- 平安只有前 3 天的记录；后 3 天没有对应行。

问题：怎样让日期重新成为索引、股票重新成为列？

```
restored = long_prices.pivot(
    index="日期", columns="股票", values="收盘价")
restored = restored.reindex(columns=prices_wide.columns)
print(restored)
```

日期	茅台	五粮液	平安
2026-07-01	1500.0	128.0	55.0
2026-07-02	1520.0	129.5	54.5
2026-07-03	1515.0	127.0	56.0
2026-07-06	1530.0	130.0	NaN
2026-07-07	1525.0	129.0	NaN
2026-07-08	1545.0	131.2	NaN

`index/columns/values` 分别指定行标签、列标签、单元格取值；`reindex` 恢复展示列顺序。`pivot` 不计算均值或总和，要求每个“日期–股票”组合唯一。

	宽表	长表
一行的含义	一个日期，多只股票并排	一只股票在一个日期的记录
适合查看	同一天各股票的价格	某只股票的全部观测记录
适合计算	按列计算收益率、相关矩阵；价格矩阵运算	按股票或日期分组；组内筛选、自定义计算
适合整理	固定股票集合的时间序列表	追加股票记录、按股票和日期连接其他信息
缺少报价	对应单元格通常显示 NaN	可以是含 NaN 的行，也可以没有该行，需说明口径

本章接下来的选择

使用长表 `market` 演示按股票分组；每组仍是一只股票按日期排列的价格记录。需要价格矩阵时，再用 `pivot` 转回宽表。

6. 排序与分组

问题：怎样把同一只股票的记录放在一起，并按时间先后排列？

```
market = market.sort_values(["股票", "日期"])
print(market.head(3))
#     股票          日期     收盘价
# 6  五粮液 2026-07-01  128.0
# 7  五粮液 2026-07-02  129.5
# 8  五粮液 2026-07-03  127.0
```

- 排序键按先后生效：先按股票名排序，同一股票内再按日期升序。
- 日期在这里是**普通列**，所以用 `sort_values`；`sort_index` 排的是左侧行索引。
- 输出左侧的 6、7、8 是保留的原行索引，不是重新编号。
- 排序返回新表，通过赋值更新 `market`。后面的首末价格计算都使用这一日期顺序。

问题：每只股票在已有记录中的平均收盘价是多少？

```
print(market.groupby("股票")["收盘价"].mean().round(2))
# 股票
# 五粮液      129.12
# 平安         55.17
# 茅台       1522.50
# Name: 收盘价, dtype: float64
```

1. `groupby("股票")`：同一股票在不同日期的记录归入一组。
2. `["收盘价"]`：指定要计算的数值列。
3. `mean()`：各组分别求均值；`round(2)` 将结果保留两位小数。

从每日记录到股票汇总

输入 15 行行情，输出 3 个股票均值。茅台、五粮液各用 6 天，平安仅用 3 天；这里是各自已有记录的价格均值。

agg：同时查看数量、均值与范围

问题：均值由多少天计算？各只股票的价格范围是多少？

```
result = market.groupby("股票")["收盘价"].agg(
    ["count", "mean", "min", "max"])
print(result.round(2))
```

股票	count	mean	min	max
五粮液	6	129.12	127.0	131.2
平安	3	55.17	54.5	56.0
茅台	6	1522.50	1500.0	1545.0

- agg 接受多个聚合方法名；每只股票一行，每个指标一列。
- count 统计有效价格数量；本例每天每只股票至多一行，因此也是有报价的日期数。
- min/max 给出组内最低、最高收盘价，单位均为元。

问题：怎样保留“股票”列，并给汇总指标命名？

```
summary = market.groupby("股票", as_index=False).agg(
    记录数=("日期", "size"),
    平均收盘价=("收盘价", "mean"))
print(summary.round(2))
```

	股票	记录数	平均收盘价
0	五粮液	6	129.12
1	平安	3	55.17
2	茅台	6	1522.50

- 写法为**新列名 = (输入列名，聚合方法)**，可对不同列采用不同方法。
- `as_index=False` 让股票名成为普通列，结果使用 0、1、2 的行索引。
- `size` 统计组内记录行数；原表一行是一条行情，所以这里统计的是行情记录数。

问题：把茅台 7 月 2 日的价格设为缺失，记录数与有效价格数各是多少？

```
missing = market.copy()
missing.loc[1, "收盘价"] = float("nan")
counts = missing.groupby("股票").agg(
    记录数=("收盘价", "size"),
    有效价格数=("收盘价", "count"),
    平均收盘价=("收盘价", "mean"))
print(counts.round(2))
```

股票	记录数	有效价格数	平均收盘价
五粮液	6	6	129.12
平安	3	3	55.17
茅台	6	5	1523.00

`loc[1]` 使用保留的原行标签，对应茅台 7 月 2 日。`size` 数所有行，`count` 数非缺失价格；茅台均值只用剩下 5 个有效价格。平安则是后 3 天的记录本身没有出现。

问题：每一天的收盘价，比这只股票的样本均价高多少？

```
relative = market.copy()
relative["样本均价"] = relative.groupby("股票")[
    "收盘价"].transform("mean")
relative["偏离均价"] = relative["收盘价"] - relative["样本均价"]
print(relative.loc[relative["股票"] == "茅台"].round(2))
```

	股票	日期	收盘价	样本均价	偏离均价
0	茅台	2026-07-01	1500.0	1522.5	-22.5
1	茅台	2026-07-02	1520.0	1522.5	-2.5
2	茅台	2026-07-03	1515.0	1522.5	-7.5
3	茅台	2026-07-06	1530.0	1522.5	7.5
4	茅台	2026-07-07	1525.0	1522.5	2.5
5	茅台	2026-07-08	1545.0	1522.5	22.5

agg 每只股票输出一行；transform 将均价对应回原来的 15 行，上表只展示茅台的 6 行。样本均价使用整个区间的信息，适合描述这段数据，不能作为当时已知的交易信号。

课堂练习：生成股票价格汇总表

沿用完整的 `market` 数据：

1. 每只股票输出一行，保留“股票”作为普通列。
2. 计算记录数、最高收盘价、最低收盘价，并给结果列命名。
3. 新增“价格差”列，等于最高收盘价减去最低收盘价。

先判断，再运行

每只股票用了几天的数据？价格差的单位是什么？只比较价格差的绝对大小，能否判断哪只股票的相对波动更大？

分组循环：每次拿到一只股票的小表

```python
for name, group in market.groupby("股票", sort=False):
    print(name, len(group),
          group["收盘价"].iloc[0],
          group["收盘价"].iloc[-1])
# 五粮液 6 128.0 131.2
# 平安 3 55.0 56.0
# 茅台 6 1500.0 1545.0
```

- `name` 是组名，例如“茅台”；`group` 是该股票全部记录组成的 DataFrame，保留股票、日期、收盘价三列。
- 每行输出依次为：股票名、记录数、首日价、末日价。
- `sort=False` 按组名首次出现的顺序遍历；组内行顺序沿用输入，本节开头已按股票和日期排序。

什么时候用循环?

适合逐组查看数据、画图或保存结果。只是计算均值、最大值等常规统计时，直接使用聚合方法更简洁。

问题：只研究至少有 5 条价格记录的股票，如何保留它们的全部行情？

```
kept = market.groupby("股票", sort=False).filter(
    lambda group: len(group) >= 5)
print(kept.groupby("股票", sort=False).size())
# 股票
# 五粮液    6
# 茅台      6
# dtype: int64
print(kept.shape)
# (12, 3)
```

- `lambda group: ...` 接收一组数据，返回一个布尔值：这一组是否满足条件。
- 茅台、五粮液各有 6 条，保留全部记录；平安仅 3 条，整组删除。
- 输出仍是原来的三列、共 12 行行情。上方的 `size()` 只是用来检查筛选结果。

区别：布尔索引按每行的条件筛选；`GroupBy.filter` 按整组的条件决定去留。本例数的是记录行数，不是验证所有应有交易日都已齐全。

问题：把“取首末价格并计算涨幅”写成函数，如何分别应用于保留下来的股票？

```python
def price_summary(group):
    prices = group["收盘价"]
    return pd.Series({
        "首日价": prices.iloc[0],
        "末日价": prices.iloc[-1],
        "区间涨幅(%)": (prices.iloc[-1] / prices.iloc[0] - 1) * 100})

result = kept.groupby("股票", sort=False)[["收盘价"]].apply(
    price_summary)
print(result.round(2))
```

股票	首日价	末日价	区间涨幅 (%)
五粮液	128.0	131.2	2.5
茅台	1500.0	1545.0	3.0

每组传入一列价格组成的 DataFrame；函数返回带指标名称的 Series，apply 将它们合成汇总表。本例两组均覆盖 7 月 1–8 日；涨幅是首末价格变化，未计分红与交易成本。

自定义 apply 很灵活；如果计算能拆成现成的聚合方法，也可以直接写：

```
result = kept.groupby("股票", sort=False).agg(
    首日价=("收盘价", "first"),
    末日价=("收盘价", "last"))
result["区间涨幅(%)"] = (
    result["末日价"] / result["首日价"] - 1) * 100
print(result.round(2))
```

股票	首日价	末日价	区间涨幅 (%)
五粮液	128.0	131.2	2.5
茅台	1500.0	1545.0	3.0

本例价格无缺失，两种写法结果相同。first/last 默认取组内首末非缺失值，而 iloc[0]/iloc[-1] 取实际首末位置；有缺失时需明确口径。两种方法都依赖事先排好的日期顺序。

分组操作：根据需要的结果选择方法

方式	返回什么	股票案例
分组循环	每次一个组名和一张组内表	逐只股票查看或绘图
聚合（agg）	每组一行汇总指标	每只股票的首日价、末日价
转换（transform）	与原数据等长且索引对齐的结果	每条行情对应的该股票均价
筛选（filter）	满足组条件的原始记录	保留记录数至少为 5 的股票
应用（apply）	合并自定义函数的返回值，形状由函数决定	自定义计算每组的价格摘要

课堂练习

把筛选条件改为“至少 3 条记录”，再计算每只股票的区间涨幅。平安的结果对应哪段日期？能否直接与另外两只股票的涨幅比较？

7. 数据清洗基础与缺失值

问题：怎样分别处理报价表中的文本、缺失、重复和异常？下面是故意含有问题的教学数据，每行是一条报价登记。

```
raw_quotes = pd.DataFrame({
    "代码": [" 600519 ", "600519", "000858", "601318", "600036"],
    "日期": ["2026-07-01"] * 4 + ["待核"],
    "价格文本": ["1500", "1500", "未披露", "-5", "40"]})
print(raw_quotes)
```

	代码	日期	价格文本
0	␣600519␣	2026-07-01	1500
1	600519	2026-07-01	1500
2	000858	2026-07-01	未披露
3	601318	2026-07-01	-5
4	600036	待核	40

表中首尾空格以可见符号标示。代码保留为字符串，避免丢失前导 0；价格和日期先保留原文，便于核对处理依据。

字符串清理：str 对每个元素执行操作

```python
quotes_clean = raw_quotes.copy()
quotes_clean["代码"] = quotes_clean["代码"].str.strip()
print(quotes_clean["代码"].tolist())
# ['600519', '600519', '000858', '601318', '600036']
print(quotes_clean["代码"].str.startswith("60").tolist())
# [True, True, False, True, True]
```

- `str.strip()` 删除每个字符串首尾的空白，统一代码写法。
- `str.startswith("60")` 逐个判断前缀，返回布尔结果，可用于筛选。
- `str.upper()` 可统一英文字母大小写；`str.contains()` 可查找指定文本。

先统一，再识别

前两条代码只差首尾空格，统一后才能准确识别重复。`raw_quotes` 保留原始文本。

类型转换：区分已知缺失与解析失败

```
price_text = quotes_clean["价格文本"].replace("未披露", pd.NA)
quotes_clean["价格"] = pd.to_numeric(price_text, errors="coerce")
quotes_clean["交易日"] = pd.to_datetime(
    quotes_clean["日期"], errors="coerce")
failed = price_text.notna() & quotes_clean["价格"].isna()
print(failed.tolist())
# [False, False, False, False, False]
print(quotes_clean[["价格", "交易日"]])
```

	价格	交易日
0	1500.0	2026-07-01
1	1500.0	2026-07-01
2	NaN	2026-07-01
3	-5.0	2026-07-01
4	40.0	NaT

`coerce` 将无法解析的文本转为缺失；日期缺失显示为 `NaT`。“未披露”已明确作为缺失处理，-5 能成功转为数值，但仍需业务规则检查。

重复记录：整行重复与键冲突不同

```python
print(quotes_clean.duplicated().tolist())
# [False, True, False, False, False]
print(quotes_clean.duplicated(
    ["代码", "日期"], keep=False).tolist())
# [True, True, False, False, False]
quotes_clean = quotes_clean.drop_duplicates()
print(quotes_clean.index.tolist())
# [0, 2, 3, 4]
```

- `duplicated()` 默认比较所有列，标记重复记录中除第一次以外的行。
- 指定“代码、日期”检查组合键；`keep=False` 标记冲突中的所有行。
- 本例确认前两条是同一次报价的重复登记，删除整行重复后保留 4 条。

不能直接删除的情况

若同一股票、同一日期出现不同价格，应先核查来源或报价时点，不能随意按组合键保留第一条。

本例是股票收盘价，非正价格需要核查。保留问题标记，单独生成用于统计的有效价格列。

```
quotes_clean["价格异常"] = quotes_clean["价格"].le(0)
quotes_clean["有效价格"] = quotes_clean["价格"].mask(
    quotes_clean["价格异常"])
print(quotes_clean[["代码", "价格异常", "有效价格"]])
```

	代码	价格异常	有效价格
0	600519	False	1500.0
2	000858	False	NaN
3	601318	True	NaN
4	600036	False	40.0

`mask` 将条件为真的位置替换为缺失。未披露与异常价格在统计列中都显示为缺失，但原因不同；日期待核的 40 元报价也不能直接用于按日分析。

```python
quotes_clean["名称"] = quotes_clean["代码"].map({
    "600519": "茅台", "000858": "五粮液",
    "601318": "平安", "600036": "招行"})
quotes_clean["价格区间"] = pd.cut(
    quotes_clean["有效价格"], bins=[0, 100, 1000, float("inf")],
    right=False, labels=["低于100", "100至不足1000", "至少1000"])
print(quotes_clean[["名称", "价格区间"]])
```

	名称	价格区间
0	茅台	至少 1000
2	五粮液	NaN
3	平安	NaN
4	招行	低于 100

`map` 按字典映射，未找到对应键时为缺失；`cut` 按给定边界分段，`right=False` 表示左闭右开。标签仅描述价格水平，不是估值高低判断。

```python
import numpy as np
s = pd.Series([1500.0, np.nan, 1515.0])

s.isna()          # [False, True, False]  找出缺失
s.dropna()        # 删掉缺失的行
s.fillna(s.mean())    # 用均值填充
s.ffill()         # 用前一个有效值填充（金融常用！）
```

- NaN（Not a Number）表示缺失：停牌、数据未披露、pct_change 的第一行。
- ffill()（前向填充）：沿用上次观测值；是否适合需依据数据含义判断。
- 处理前先想：缺失代表什么经济含义？乱填会引入偏差。

问题：合并两个账户的持股数量。已确认清单完整，未列出的股票表示该账户持有 0 股。

```
account_a = pd.Series({"茅台": 100, "平安": 200})
account_b = pd.Series({"平安": 300, "招行": 400})
total = account_a.add(account_b, fill_value=0)
print(total.reindex(["茅台", "平安", "招行"]))
# 茅台    100.0
# 平安    500.0
# 招行    400.0
# dtype: float64
```

- `fill_value=0` 在相加前，将一方缺少的对应值按 0 处理。
- 茅台：$100 + 0 = 100$ 股；平安：$200 + 300 = 500$ 股；招行：$0 + 400 = 400$ 股。
- `reindex` 仅指定结果展示的股票顺序。

缺失价格不能照此处理

未取得报价表示价格未知，不表示价格为 0。只有业务含义支持时才补零；双方都缺失的位置仍为缺失值。

8. 数据合并与变形进阶

合成订单表每行一笔订单，商品表每行一种商品。同一商品可以出现在多笔订单中，因此合并关系是多对一。

```
import pandas as pd
orders = pd.DataFrame({
    "订单": [1, 2, 3, 4], "商品": ["T", "C", "T", "X"],
    "数量": [2, 1, 3, 1]})
products = pd.DataFrame({
    "商品": ["T", "C", "J"],
    "名称": ["茶", "咖啡", "果汁"], "单价": [10, 20, 15]})
print(orders)
print(products)
```

```
   订单 商品  数量
0   1  T   2
1   2  C   1
2   3  T   3
3   4  X   1
  商品  名称  单价
0  T   茶  10
1  C  咖啡  20
2  J  果汁  15
```

订单中的 T 出现两次；商品表中的 T 只出现一次。X 暂时没有商品信息，J 暂时没有订单。

on 明确匹配键；left 保留左表订单。indicator 标记未匹配行，validate 检查商品表的键唯一，避免合并后订单被复制。

```
joined = orders.merge(products, on="商品", how="left",
    validate="many_to_one", indicator=True)
print(joined)
print(joined["_merge"].value_counts())
print("合并前后行数: ", len(orders), len(joined))
```

```
   订单 商品  数量   名称    单价      _merge
0   1  T   2    茶  10.0        both
1   2  C   1   咖啡  20.0        both
2   3  T   3    茶  10.0        both
3   4  X   1  NaN   NaN  left_only
_merge
both          3
left_only     1
right_only    0
Name: count, dtype: int64
合并前后行数:  4 4
```

4 笔订单全部保留。X 的名称与单价缺失，标记为 left_only；其余 3 行为 both。right_only 为 0，因为左连接不保留仅右表出现的 J。

inner 只保留双方存在的商品匹配；outer 还会显示没有订单的商品。未匹配不代表金额为零，应单独列出。

```
inner = orders.merge(products, on="商品", how="inner")
outer = orders.merge(products, on="商品", how="outer",
                     indicator=True)
print("内连接、外连接行数: ", len(inner), len(outer))
print(inner[["订单", "商品"]])
print(outer[["订单", "商品", "_merge"]])
```

```
内连接、外连接行数:  3 5
   订单 商品
0   1  T
1   2  C
2   3  T
    订单 商品      _merge
0  2.0  C        both
1  NaN  J  right_only
2  1.0  T        both
3  3.0  T        both
4  4.0  X   left_only
```

inner 中订单 4（X）消失；outer 中 X 保留，并新增仅在右表出现的 J，其订单号为 NaN。

哪笔订单在 inner 中消失？哪个商品只在右表中？如果要核对全部订单，该选哪种连接？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

在商品表中故意重复 T，观察两笔 T 订单各匹配两次。解决方法是核实键与版本，而不是合并后随意去重。

```
bad_products = pd.concat([products, products.iloc[[0]]],
                         ignore_index=True)
expanded = orders.merge(bad_products, on="商品", how="left")
print("合并前后行数: ", len(orders), len(expanded))
print(expanded[["订单", "商品", "数量", "单价"]])
print("重复商品键数: ", bad_products.duplicated("商品").sum())
# 加 validate="many_to_one" 将直接报错，课堂可自行尝试。
```

```
合并前后行数:  4 6
   订单 商品  数量    单价
0   1  T   2  10.0
1   1  T   2  10.0
2   2  C   1  20.0
3   3  T   3  10.0
4   3  T   3  10.0
5   4  X   1   NaN
重复商品键数:  1
```

订单 1、3 都购买 T，各匹配到两条商品记录，因此各出现两次：4 行变为 6 行。

concat 不按商品键寻找匹配。ignore_index 重建行号，不检查业务编号是否重复；拼接后仍要检查订单号。

```
next_orders = pd.DataFrame({
    "订单": [5, 6], "商品": ["C", "T"], "数量": [2, 1]})
all_orders = pd.concat([orders, next_orders], ignore_index=True)
print(all_orders)
print(all_orders["订单"].is_unique)
```

```
   订单 商品  数量
0   1  T   2
1   2  C   1
2   3  T   3
3   4  X   1
4   5  C   2
5   6  T   1
True
```

前 4 行来自原订单，后 2 行来自新增订单；True 表示订单号仍唯一。

axis=1 横向拼接时按索引标签对齐，不会因为行数相同就按位置贴在一起。先检查索引含义。

```
left = pd.Series([80, 90], index=["001", "002"], name="数学")
right = pd.Series([70, 85], index=["002", "001"], name="英语")
print(pd.concat([left, right], axis=1))
```

```
     数学  英语
001  80  85
002  90  70
```

001 的数学 80 与英语 85 对齐；002 的数学 90 与英语 70 对齐。右侧 Series 的原排列顺序不决定拼接位置。

加入一次补考后，同一学生同一课程出现两次。pivot_table 可以聚合；取最大值代表“保留最高成绩”，这是业务规则。

```
long_scores = pd.DataFrame({
    "学号": ["001", "001", "002", "002"],
    "课程": ["数学", "英语", "数学", "英语"],
    "成绩": [80, 90, 70, 85]})
retake = pd.DataFrame({"学号": ["002"],
                       "课程": ["数学"], "成绩": [88]})
attempts = pd.concat([long_scores, retake], ignore_index=True)
best = attempts.pivot_table(index="学号", columns="课程",
    values="成绩", aggfunc="max")
print(best)
```

```
课程    数学  英语
学号
001   80   90
002   88   85
```

002 的数学有 70、88 两次记录，max 得到 88；001 的数学仍为 80。这里的值来自明确的聚合规则。

将 max 改为 mean，002 的数学成绩变成多少？两种答案分别回答什么问题？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

MultiIndex 可以表示“学号—课程”这类组合标签。先学 set_index 和 reset_index，再选学 stack、unstack。

```
indexed = long_scores.set_index(["学号", "课程"]).sort_index()
print(indexed)
print(indexed.loc[("001", "数学"), "成绩"])
print(indexed.unstack("课程"))
print(indexed.reset_index())
```

```
          成绩
学号   课程
001  数学   80
     英语   90
002  数学   70
     英语   85
80
      成绩
课程    数学   英语
学号
001   80   90
002   70   85
     学号   课程   成绩
0   001   数学   80
1   001   英语   90
2   002   数学   70
3   002   英语   85
```

先显示组合索引表；用（001，数学）定位得到 80。unstack 将课程移到列上，reset_index 将索引还原为普通列。

价格未知的 X 订单不能当作零金额。先列出未匹配订单，再汇总可计算的部分，并说明覆盖范围。

```
matched = joined.loc[joined["_merge"] == "both"].copy()
matched["金额"] = matched["数量"] * matched["单价"]
print(matched[["订单", "名称", "金额"]])
print("已知金额合计：", matched["金额"].sum())  # 70
print("未匹配订单数：", (joined["_merge"] == "left_only").sum())
```

```
   订单  名称    金额
0   1   茶  20.0
1   2  咖啡  20.0
2   3   茶  30.0
已知金额合计：  70.0
未匹配订单数：  1
```

已知金额合计 70 元仅覆盖匹配成功的 3 笔订单；X 的 1 笔订单仍待补充单价，不能把 70 元称为全部订单总额。

将商品 X 补录为“水”、单价 5，重新合并全部原订单。总金额应是多少？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

9. 金融综合应用：行情与样本检查

案例任务：从行情表到可比的收益率样本

沿用茅台、五粮液、平安在 7 月 1–8 日的教学行情，不另建数据集。

- `prices_wide`：6 个日期 × 3 只股票，保留缺失价格。
- `long_prices`：18 行“日期–股票”组合，包括平安后 3 天的缺失报价。
- `market`：15 条有效报价，适合查看已知价格，但不保留完整的日期组合。

要完成的分析

检查每天的数据覆盖情况；在各只股票内部计算相邻观测日收益率；整理收益率矩阵，并确定三只股票可共同比较的日期。

本案例使用 `long_prices` 保留缺失位置。提前删掉缺失价格后再错位，可能把跨多日的价格变化误认为相邻观测日变化。

行情覆盖：每天是否都有三只股票的报价？

```
coverage = long_prices.groupby("日期").agg(
    应有股票数=("股票", "size"), 有效报价数=("收盘价", "count"))
coverage["缺失报价数"] = coverage["应有股票数"] - coverage["有效报价数"]
print(coverage)
```

日期	应有股票数	有效报价数	缺失报价数
2026-07-01	3	3	0
2026-07-02	3	3	0
2026-07-03	3	3	0
2026-07-06	3	2	1
2026-07-07	3	2	1
2026-07-08	3	2	1

- 后 3 天各缺少 1 个报价，不能把剩余两只股票当作完整的三股票样本。
- 这里能用 `size` 表示应有数量，是因为宽表展开时保留了全部 18 个日期–股票组合；在 `market` 上计数只能得到有效报价数。

组内错位：价格必须与同一股票的前期价格配对

```
quote_panel = long_prices.sort_values(["股票", "日期"]).copy()
quote_panel["上一观测日价"] = quote_panel.groupby("股票")["收盘价"].shift(1)
quote_panel["收益率"] = (
    quote_panel["收盘价"] / quote_panel["上一观测日价"] - 1)
print(quote_panel.loc[quote_panel["股票"] == "平安",
    ["日期", "收盘价", "上一观测日价", "收益率"]].round(4))
```

	日期	收盘价	上一观测日价	收益率
12	2026-07-01	55.0	NaN	NaN
13	2026-07-02	54.5	55.0	-0.0091
14	2026-07-03	56.0	54.5	0.0275
15	2026-07-06	NaN	56.0	NaN
16	2026-07-07	NaN	NaN	NaN
17	2026-07-08	NaN	NaN	NaN

`shift(1)` 在每只股票组内取上一行价格；收益率为 $P_t/P_{t-1}-1$，上表用小数表示。首日没有前期价格；任一端缺失，收益率也缺失。观测日期不是逐个日历日，不能将跨周末变化称为单个日历日变化。

收益率矩阵：同一日期横向查看三只股票

```python
returns = quote_panel.pivot(
    index="日期", columns="股票", values="收益率")
returns = returns.reindex(columns=prices_wide.columns)
print((returns * 100).round(2))
```

日期	茅台	五粮液	平安
2026-07-01	NaN	NaN	NaN
2026-07-02	1.33	1.17	-0.91
2026-07-03	-0.33	-1.93	2.75
2026-07-06	0.99	2.36	NaN
2026-07-07	-0.33	-0.77	NaN
2026-07-08	1.31	1.71	NaN

- 输出单位为百分比；例如茅台 7 月 2 日上涨约 1.33%。`returns` 本身仍保存小数收益率。
- 价格缺失产生的收益率缺失继续保留，没有填成 0，也没有沿用前一个收益率。

问题：若各列直接跳过缺失值，三只股票的统计是否使用了同一段观测？

```
print(returns.count().tolist())
# [5, 5, 2]
common_returns = returns.dropna(how="any")
print((common_returns * 100).round(2))
```

日期	茅台	五粮液	平安
2026-07-02	1.33	1.17	-0.91
2026-07-03	-0.33	-1.93	2.75

- 原矩阵有效收益率数依次为 5、5、2，各列直接取均值会使用不同样本。
- `dropna(how="any")` 保留三列均有值的行，得到 2 个共同日期；原矩阵 `returns` 不变。
- 共同样本便于同日期比较，但会损失数据。这里仅剩两条观测，只用于演示方法，不据此评估长期风险。

课堂练习：增加报价后，哪些结果会改变？

1. 在 `prices_wide` 的副本中，为平安补入经核实的 7 月 6 日报价 55.5 元；其余报价保持原样。
2. 从宽表重新生成长表，更新覆盖表、组内收益率和收益率矩阵。
3. 手算平安 7 月 6 日相对于 7 月 3 日的价格收益率，再与代码核对。
4. 检查共同样本新增了哪个日期。7 月 7 日是否也能计算收益率？说明原因。

交付结果

一张覆盖表、一张收益率矩阵，以及共同样本的日期列表。解释每个新增有效值来自哪两个日期的报价。

10. 金融时间序列

沿用前面的教学合成行情，每行一个观测日期，每列一只股票，价格单位为元。

```
pd.set_option("display.unicode.east_asian_width", True)
ts_prices = prices_wide.copy()
print(ts_prices.to_string())
```

```
            茅台   五粮液   平安
日期
2026-07-01  1500   128.0  55.0
2026-07-02  1520   129.5  54.5
2026-07-03  1515   127.0  56.0
2026-07-06  1530   130.0   NaN
2026-07-07  1525   129.0   NaN
2026-07-08  1545   131.2   NaN
```

索引已经是 DatetimeIndex，并按日期升序排列。7 月 3 日之后是 7 月 6 日；平安后 3 个观测日缺报价。本节保留这些缺失值。

日期字符串转换后才能参与时间运算；工作日序列可以用于检查日期覆盖。

```
print(pd.to_datetime(["2026-07-03", "2026-07-06"]))
bdays = pd.date_range("2026-07-01", periods=6, freq="B")
print(bdays.strftime("%Y-%m-%d").tolist())
print(ts_prices.loc["2026-07-02":"2026-07-03"].to_string())
```

```
DatetimeIndex(['2026-07-03', '2026-07-06'], dtype='datetime64[ns]', freq=None)
['2026-07-01', '2026-07-02', '2026-07-03', '2026-07-06', '2026-07-07', '2026-07-08']
            茅台   五粮液   平安
日期
2026-07-02  1520   129.5  54.5
2026-07-03  1515   127.0  56.0
```

B 表示周一至周五，不排除交易所节假日；实际交易日应以交易所日历为准。日期标签切片包含两端。

时间点用 Timestamp 表示；两个时间点之差是 Timedelta，即经过的时间长度。

```
gap = pd.Timestamp("2026-07-06") - pd.Timestamp("2026-07-03")
print(gap)
print(pd.Timestamp("2026-07-03") + pd.Timedelta(days=1))
gaps = ts_prices.index.to_series().diff()
print(gaps.to_string())
```

```
3 days 00:00:00
2026-07-04 00:00:00
日期
2026-07-01      NaT
2026-07-02   1 days
2026-07-03   1 days
2026-07-06   3 days
2026-07-07   1 days
2026-07-08   1 days
```

周五到周一相隔 3 个日历日，但在本表中只跨一个观测间隔。`Timedelta(days=1)` 加一天会到周六，并不等于下一个交易日。

比较前 3 条茅台报价：上一行价格与把日期向后推一天，是不同操作。

```
s = ts_prices["茅台"].head(3)
print(pd.DataFrame({"原价": s, "上一行价格": s.shift(1)}).to_string())
print(s.shift(1, freq="D").to_string())
```

```
            原价  上一行价格
日期
2026-07-01  1500        NaN
2026-07-02  1520     1500.0
2026-07-03  1515     1520.0
日期
2026-07-02    1500
2026-07-03    1520
2026-07-04    1515
```

`shift(1)` 保留索引、移动数值，首行缺失；`shift(1, freq="D")` 保留数值、移动日期标签，最后一个标签变成周六。计算相邻观测收益率用前者。

diff 给出价格变动额（元）；pct_change 给出相对于上一观测价格的变化比例。

```
changes = pd.DataFrame({
    "茅台价差": ts_prices["茅台"].diff(),
    "茅台收益率(%)": ts_prices["茅台"].pct_change(fill_method=None) * 100,
    "平安收益率(%)": ts_prices["平安"].pct_change(fill_method=None) * 100})
print(changes.round(2).to_string())
```

```
            茅台价差  茅台收益率(%)  平安收益率(%)
日期
2026-07-01      NaN         NaN         NaN
2026-07-02     20.0        1.33       -0.91
2026-07-03     -5.0       -0.33        2.75
2026-07-06     15.0        0.99         NaN
2026-07-07     -5.0       -0.33         NaN
2026-07-08     20.0        1.31         NaN
```

整表计算写作 `ts_prices.pct_change(fill_method=None)`。首行无前值；任一期价格缺失时收益率也缺失。7 月 6 日的收益率跨周末，不是单个日历日的收益率。

下采样：把多条日度报价汇总为一周

下采样把较高频率数据汇总到较低频率。以周五作为每周结束标签，先比较期末价格和平均价格。

```
weekly_last = ts_prices.resample("W-FRI").last()
print(weekly_last.to_string())
print(ts_prices.resample("W-FRI").mean().round(2).to_string())
```

```
            茅台   五粮液   平安
日期
2026-07-03  1515   127.0  56.0
2026-07-10  1545   131.2   NaN
              茅台   五粮液    平安
日期
2026-07-03  1511.67  128.17  55.17
2026-07-10  1533.33  130.07    NaN
```

W-FRI 默认右闭、标记在周五：7 月 3 日所在组包括截至该日的一周。7 月 10 日只是分组标签，本样本实际仅到 7 月 8 日，因此第二组不是完整一周。

下采样选什么聚合：先看业务含义

每周有几条有效报价？用日收盘价还能汇总出什么？

```
print(ts_prices.resample("W-FRI").count().to_string())
print(ts_prices["茅台"].resample("W-FRI").ohlc().to_string())
```

```
            茅台  五粮液  平安
日期
2026-07-03     3       3     3
2026-07-10     3       3     0
            open  high   low  close
日期
2026-07-03  1500  1520  1500   1515
2026-07-10  1530  1545  1525   1545
```

last 取每列最后一个非缺失值，mean 取均值，count 数有效值。本例 ohlc 的四列是周内日收盘价的首值、最大、最小、末值，不是完整周 K 线；真正周 K 线需日开高低收数据。价格通常不求和，成交量才常用 sum。

期末价格的变化比例，与周内日收益率的简单平均不同。

```
weekly_ret = weekly_last.pct_change(fill_method=None)
print((weekly_ret * 100).round(2).to_string())
print(ts_prices.resample("ME").last().to_string())
```

```
            茅台   五粮液   平安
日期
2026-07-03   NaN     NaN   NaN
2026-07-10  1.98    3.31   NaN
            茅台   五粮液   平安
日期
2026-07-31  1545   131.2  56.0
```

茅台第二组为 $(1545/1515-1)\times 100\%\approx 1.98\%$，只覆盖 7 月 3 日至 8 日。月末标签 7 月 31 日对应的也是样本内最后有效报价，不是真实月末价格。只有一个月分组，无法计算月间收益率。

课堂练习：扩充同一份行情，观察周频结果

1. 在行情副本中补入 7 月 9 日和 10 日的教学报价：茅台 1540、1550；五粮液 130.5、132；平安 56.5、57。
2. 打印日期间隔，解释新增观测的 Timedelta。
3. 重新输出周末价格、有效报价数和周频收益率。
4. 比较扩充前后的第二周结果：日期标签是否改变？最后有效价格对应的日期是否改变？

提交与解释

提交代码和输出。指出平安第二周参与汇总的报价数量，并解释：每周有期末报价，是否意味着周内每天都有报价？

11. 读写数据

```python
df = pd.read_csv("../data/eod_data.csv",        # 文件路径
                 index_col=0,                   # 第0列作为索引
                 parse_dates=True)              # 把索引解析为日期
```

- 真实行情/财报数据多以 CSV、Excel 形式存在。
- 读入时顺手完成"第一列当日期索引"，后面全受益。
- 导出同样简单：`df.to_csv("../outputs/result.csv")`。
- Excel：`pd.read_excel("../data/data.xlsx")`（需安装 openpyxl）。

路径小贴士

从 `notebooks/` 运行时，`../` 表示本章目录；
输入放 `data/`，生成结果放 `outputs/`。

12. 综合案例与小结

- 输入：6 条原始登记，包含文本差异、重复、缺考、异常分数和待核日期。
- 任务：应用字符串处理、类型转换、重复检查、异常标记及缺失处理，形成可解释的统计结果。
- 输出：保留记录、有效成绩、问题标记与清洗汇总；最后检查导出再读回是否保留学号。

练习方式

先根据原始表判断问题，再逐步运行并核对结果。配套练习与折叠参考答案见 `04_pandas` 第 12 节。

教学合成数据，每行一次成绩登记。故意保留首尾空格、重复记录、缺考和不合理成绩；学号按字符串读入以保留前导零。

```
from io import StringIO
import pandas as pd
raw_csv = """学号,姓名,班级,成绩,日期
001, 小林 ,A,82,2026-09-01
002,小周,a,95,2026-09-01
003,小陈,B,缺考,2026-09-02
004,小许,B,108,2026-09-02
001, 小林 ,A,82,2026-09-01
005,小吴, B ,76,日期待核"""
raw = pd.read_csv(StringIO(raw_csv), dtype="string")
raw
```

info 看类型和非缺失数，duplicated 找整行重复。这里“缺考”是普通文本，尚不会被 isna 自动识别。

```
raw.info()
print(raw.isna().sum())
print("整行重复数: ", raw.duplicated().sum())
raw.loc[raw.duplicated(keep=False)]
```

copy 保留原始登记。统一班级格式后再分组，否则 A 和 a 会被算成两个班。

```
clean = raw.copy()
clean["姓名"] = clean["姓名"].str.strip()
clean["班级"] = clean["班级"].str.strip().str.upper()
print(clean["班级"].value_counts())
print(clean["姓名"].str.contains("小", na=False))
```

先把明确的缺考编码改成缺失，再转数值。coerce 将无法解析的值变成缺失；保留原列才能区分缺考与格式错误。

```
score_text = clean["成绩"].replace("缺考", pd.NA)
clean["分数"] = pd.to_numeric(score_text, errors="coerce")
clean["登记日"] = pd.to_datetime(clean["日期"], errors="coerce")
parse_failed = score_text.notna() & clean["分数"].isna()
print(clean.loc[parse_failed | clean["登记日"].isna()])
print(clean.dtypes)
```

本例每名学生只有一次考试，学号应唯一。已确认整行相同的重复登记可以删除；若同一学号分数不同，应先核实，不能任意保留。

```
print(clean.loc[clean.duplicated("学号", keep=False)])
clean = clean.drop_duplicates().reset_index(drop=True)
print("清理后记录数: ", len(clean))
print("学号唯一: ", clean["学号"].is_unique)
```

考试满分 100，因此 108 是待核异常，不能直接截成 100。缺考也不等于 0 分。设置标记后，在统计列中排除异常。

```
clean["成绩异常"] = (clean["分数"].notna() &
                        ~clean["分数"].between(0, 100))
clean["有效成绩"] = clean["分数"].mask(clean["成绩异常"])
print(clean[["学号", "成绩", "成绩异常", "有效成绩"]])
print("有效成绩均值: ", clean["有效成绩"].mean())
```

计算有效成绩数量、缺考数量、异常成绩数量。为什么不能把三者混成一类？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

比较同一列的有效值平均与填零平均。dropna 的 subset 明确本次分析依赖哪些列；不能因为日期缺失就删除不依赖日期的成绩。

```
print(clean["有效成绩"].mean())
print(clean["有效成绩"].fillna(0).mean())
usable_scores = clean.dropna(subset=["有效成绩"])
print(usable_scores[["学号", "有效成绩"]])
```

cut 按固定阈值分段，right=False 表示左闭右开。100 包含在 [90,101) 内；没有有效成绩的行仍保留缺失等级。

```
clean["等级"] = pd.cut(clean["有效成绩"],
    bins=[0, 60, 80, 90, 101], right=False,
    labels=["待提高", "合格", "良好", "优秀"])
clean["班级名称"] = clean["班级"].map({"A": "一班", "B": "二班"})
clean[["学号", "有效成绩", "等级", "班级名称"]]
```

先在内存中演示 CSV 往返。读回时显式指定学号类型；分类类型和日期类型需要重建。真实文件路径可替换 StringIO。

```
csv_text = clean.to_csv(index=False)
restored = pd.read_csv(StringIO(csv_text), dtype={"学号": "string"})
print(restored["学号"].tolist())
print(len(raw), len(clean), len(restored))
assert len(raw) == 6 and len(clean) == 5
assert restored["学号"].iloc[0] == "001"
```

交付三项结果：保留几行、排除哪些成绩、哪些日期待核。结合清洗依据解释各项数量，并检查结果是否符合原始登记。

```
audit = pd.Series({
    "输入行数": len(raw), "保留行数": len(clean),
    "有效成绩数": clean["有效成绩"].count(),
    "异常成绩数": clean["成绩异常"].sum(),
    "待核日期数": clean["登记日"].isna().sum()})
print(audit)
```

找出有效成绩低于 80 的学生，保留学号、姓名、班级、有效成绩。解释为什么没有把缺考者选进来。

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

```python
rng = np.random.default_rng(42)
dates = pd.date_range("2026-01-05", periods=250, freq="B")

# 模拟三只股票的日对数收益率，再还原为价格
rets = pd.DataFrame({
    "茅台": 0.0004 + 0.015 * rng.standard_normal(250),
    "平安": 0.0002 + 0.020 * rng.standard_normal(250),
    "招行": 0.0003 + 0.018 * rng.standard_normal(250)},
    index=dates)
prices = 100 * np.exp(rets.cumsum())
prices.tail(3)          # 查看最后3天
```

- 用 DataFrame 字典一次性构造三列，索引是简化的工作日序列。
- cumsum、exp 对整张表向量化生效——NumPy 功力再现。

```python
rets = prices.pct_change(fill_method=None).dropna()    # 日收益率

rets.describe().round(4)                               # 统计摘要
rets.mean() * 252                                      # 年化平均收益
rets.std() * np.sqrt(252)                              # 年化波动率
rets.corr().round(2)                                   # 相关系数矩阵
```

- 一个方法作用于**所有列**：这就是 DataFrame 的威力。
- **相关系数矩阵**：衡量资产间联动性，组合分散化的依据。
- 从构造数据到风险报告，没有写一个循环。

课堂讨论

相关系数接近 1 的两只股票，能互相分散风险吗？

- **两种结构**：Series（一列带标签）、DataFrame（一张表）；行 = 日期，列 = 资产。
- **选取**：`df[列名]` 取列；`loc` 按标签、`iloc` 按位置；布尔条件筛行。
- **排序分组**：`sort_values/sort_index`；`groupby` 三步走。
- **时间序列**：`to_datetime`、`pct_change`、`shift`、`resample`。
- **缺失值**：`isna/dropna/fillna/ffill`。
- **读写**：`read_csv(index_col=0, parse_dates=True)`。
- **金融工具箱**：`describe()`、年化波动率 $\sigma \times \sqrt{252}$、相关矩阵 `corr()`。

课后任务

1. 运行课堂 notebook，替换 `freq`、种子等参数观察输出。
2. 对案例数据：找出每只股票的最大单日跌幅及其日期（提示：`idxmin()` 返回最小值所在的索引）。
3. 选做：用 `resample("ME").last()` 得到月末价格，再算月度收益率。

下章预告

数字算出来了，怎么"看"得清楚？
下一章学习 **Matplotlib 可视化**：价格曲线、收益分布、K 线之外的图表语言。

本章内容改编自教材第 5 章[1]。

- Wes McKinney, *Python for Data Analysis*, 3rd ed.
- `https://wesmckinney.com/book/pandas-basics`
- 新增中文说明、合成数据与练习为本课程自行编写。
- 阅读正文理解概念；在配套 Notebook 中运行、修改、核对。

[1] 伊夫·希尔皮斯科. Python 金融大数据分析 [M]. 2nd ed. 北京 : 人民邮电出版社, 2018.



Thanks for your attention!

Q & A